HarnessFlow and MiHaCoBench: A Portable Multi-Agent Workflow Pack for AI Coding Assistants and a Harness-Level Benchmark

Hang Yu [1]

Abstract

Modern AI coding assistants are powerful but *one-shot*: given a request they edit code immediately, with no structured analysis, no self-criticism, no validation, and no memory across requests. We present **HarnessFlow**, a portable Markdown instruction pack—no runtime, no build step—that turns any compatible assistant (Claude Code, Codex, VS Code + Copilot) into a disciplined multi-agent system: it classifies each request into one of eight categories, fans out parallel analysis agents, challenges its own plan, validates with a QA pass, and records what it learned to persistent repository memory. HarnessFlow ships every workflow in three modes—GENERAL (thorough), FAST (token-efficient), and SKILL (community-skill-backed)—so users trade thoroughness against cost explicitly. On two independent benchmarks the FAST mode reaches the same successful outcome as GENERAL while spending only 39–50% of the tokens and shipping the leanest fully-documented code. To measure these effects rigorously we also built **MiHaCoBench**, a Python benchmark that scores the *harness* rather than the model: it holds the model fixed across arms, enforces grader integrity as its own correctness test, isolates each task to its specification, and targets the failure frontier where pipelines actually diverge.

1. Why a Harness?

The capability of large language models on code has advanced faster than the *process* wrapped around them. Left to their defaults, today's coding assistants behave like an overeager junior engineer: they read the prompt and start editing, skipping the steps a careful practitioner takes—surveying the codebase, weighing alternatives, stress-testing assumptions, verifying the result, and remembering context for next time. The model may be excellent; the *harness* around it is thin. This gap is increasingly the bottleneck: the same model produces markedly different outcomes depending on how its work is structured (Yang et al., 2024; Jimenez et al., 2024).

HarnessFlow closes this gap as a *drop-in instruction pack*: no runtime, no `npm install`, no build step—only Markdown. Copied into a repository, it gives any compatible assistant a structured workflow for *every* request: *classify* the prompt, *route* it to a real multi-step workflow, *analyze* the code from several angles in parallel, *challenge* the plan, *validate* the implementation, and *record* the outcome to repository memory so later requests start with real context. The same pack runs unmodified on three host tools, so a team standardizes its review discipline once regardless of which assistant each developer prefers.

This report contributes HarnessFlow's architecture and multi-agent pipeline (§2); its three workflow modes and the thoroughness–cost trade-off they expose, including a benchmark where FAST matches GENERAL at half the token budget (§3–§4); and MiHaCoBench, the instrument we built to evaluate harnesses rather than models (§5).

2. HarnessFlow Architecture

HarnessFlow is five cooperating pieces of Markdown. (1) A per-platform *router* (`CLAUDE.md`, `AGENTS.md`, or `copilot-instructions.md`) classifies each prompt. (2) Three *workflow families* under `workflow/` hold eight category instruction files each. (3) Fifteen *worker subagents* under `agents/` provide the roles the workflows orchestrate. (4) A shared *workflow contract* (`_lib/workflow_contract.md`) fixes cross-cutting rules. (5) A *philosophy* document encodes the behavioral core. Persistent *repository memory* lives under `repo_info/`.

Routing. The router reads the prompt and selects exactly one of eight categories—*Code*, *Refactor*, *Debug*, *Query*, *Correctness Check*, *Exec*, *PR Creation*, and *Initialize*. Trigger keywords are soft signals; classification is by primary

[1]HarnessFlow Project. Correspondence to: Hang Yu <hangyu8123@gmail.com>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024.

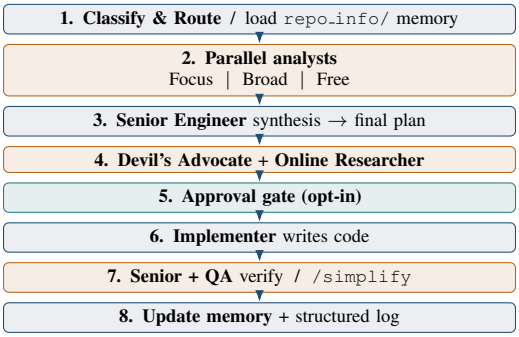

Figure 1. The GENERAL *Code* pipeline. Amber stages spawn parallel subagents. FAST executes stages 2–3 inline in the main agent and keeps a single subagent step at stage 4; SKILL additionally swaps community skills into planning, implementation, challenge, and review.

intent. An explicit `mode:` marker (or its absence) selects one of three families. The router then reads the matching `<category>.instructions.md` file in full and follows it step-by-step; multiple intents are handled sequentially.

The pipeline. Figure 1 shows the GENERAL *Code* workflow, representative of the mutating workflows. The main agent condenses repository memory into a context digest, then launches three analysts in parallel—*Focus* (only the most relevant files), *Broad* (the whole pipeline upstream to downstream), and *Free* (its own judgment). A Senior Engineer reconciles their plans; the main agent synthesizes a final plan; a *Devil's Advocate* attacks it for regressions while an *Online Researcher* checks live sources. After an opt-in approval gate, an Implementer writes the code, and a parallel Senior + QA pass verifies it before the outcome is written back to memory. The *Refactor* workflow widens the fan-out to six specialist analysts reviewed by a Principal Engineer.

Cross-cutting guarantees. The contract enforces *model parity* (subagents must never downgrade from the orchestrator's model, so quality is not silently lost in a sub-call), a *two-mode approval gate* (autonomous by default; opt-in plan-only review triggered by phrases like "plan only" or "no file changes"), and universal safety rules (never commit to GitHub, never write spam files, never use `sudo`). The philosophy document embeds four guidelines—*Think Before Coding*, *Simplicity First*, *Surgical Changes*, and *Goal-Driven Execution*—so that, by construction, every changed line traces to the request.

3. Three Workflows, One Pack

Every category ships in three modes that share the *same* skeleton—read the contract, gather context, plan, challenge, gate, implement, verify, record— but differ in how much machinery they spend (Table 1).

Table 1. The three workflow modes. "Subagent spawns" counts the dedicated parallel roles a *Code* run launches.

	GENERAL	FAST	SKILL
Planning analysts	3–6	inline	inline†
Subagent spawns (Code)	∼8	2	2†
Review stage	Senior+QA	self/native	skill/native
Approval gate	opt-in	opt-in	opt-in
Repo memory	yes	yes	yes
Relative token cost	1.0×	0.4–0.5×	≈ fast

† SKILL mirrors FAST's structure but routes eligible steps to community skills with inline fallbacks.

GENERAL (`mode: general`, the default). The thorough variant with the widest fan-out: three to six dedicated planning analysts, a Senior or Principal Engineer plan review, a Devil's Advocate / Online Researcher critique pair, and a *separate* Senior + QA verification stage that can run the full pipeline end-to-end. Use it when correctness and breadth dominate cost.

FAST (`mode: fast`). The token-efficient twin. It folds the planning analysts and the post-hoc review subagents *into the main agent*, which does analysis, planning, and implementation inline and spawns subagents in exactly one step—the Devil's Advocate + Online Researcher challenge. Verification becomes a structured self-review ("assume every change is wrong and explain why") plus native review skills and a single remediation pass. The instruction files are roughly half the size of their GENERAL counterparts, and—crucially—the empirical outcome is unchanged (§4).

SKILL (`mode: skill`). The FAST pipeline with eligible steps—planning, implementation, debugging, challenge, research, correctness—delegated to confirmed community skills, each verified at ≥ 1000 GitHub stars (e.g. `writing-plans` and `systematic-debugging` from a 229,665-star collection) and kept behind an *inline fallback* so the workflow never blocks if a skill is absent. A team rides battle-tested external tooling for generic steps while keeping HarnessFlow's structure, gates, and memory intact.

4. What Users Get

The headline benefit is that *thoroughness becomes a dial, not a gamble*. We measured the trade-off on two independent benchmarks with the *model held constant* (Sonnet-class subagents, Opus-class orchestrator) so the comparison isolates the harness, not the model: a 1,000-line greenfield OpenCV + scikit-learn build ("ShapeLab"), and a real SWE-bench bug fix (`sympy__sympy-24213`). Across both,

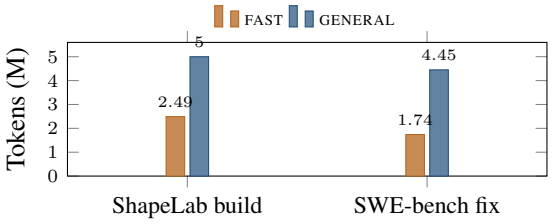

Figure 2. Token cost at *identical* outcomes. FAST reaches the same resolved/verified result as GENERAL at 2.0–2.6× lower cost.

Table 2. Quality at equal outcome (ShapeLab build; SWE-bench fix verified). The baseline is the same model with no harness.

Metric	baseline	FAST	GENERAL
Code lines (AST)	1,225	**860**	1,173
Docstring coverage	96.3%	**100%**	100%
Contract tests passed	7/10	**10/10**	10/10
Self-verified pre-ship	none	**32 green**	yes
SWE canonical fix	✗	✓	✓

FAST reached the *same successful outcome* as GENERAL while spending only 39–50% of the tokens (Figure 2) and, on ShapeLab, shipped the leanest fully-documented code of any arm (Table 2).

Beyond cost, the pack pays off in three ways. *Portability:* one instruction pack standardizes review discipline across Claude Code, Codex, and Copilot. *Quality under self-criticism:* the no-harness baseline produced a non-canonical, unverified SWE-bench patch and passed only $7/10$ contract tests, whereas both harnessed modes produced the exact canonical fix, verified, at $10/10$. *Memory and safety:* outcomes persist to `repo_info/` so the next request starts informed, while opt-in plan review and hard prohibitions on commits, spam files, and `sudo` keep autonomy bounded.

5. MiHaCoBench: Scoring the Harness

Building HarnessFlow surfaced a measurement problem: standard code benchmarks (Chen et al., 2021; Austin et al., 2021; Jain et al., 2024) score a *model* in isolation, but a harness contributes prompt handling, multi-step planning, tool use, and long-horizon consistency that those benchmarks never exercise. We therefore built **MiHaCoBench**, both the development test bench for HarnessFlow and a standalone instrument for evaluating *any* coding pipeline. Its design draws on SWE-bench, BigCodeBench, and LiveCodeBench (Jimenez et al., 2024; Zhuo et al., 2024; Jain et al., 2024). Four properties distinguish it.

1. It scores the harness, not the model. Every arm—no-harness baseline, FAST, SKILL, GENERAL—runs the same tasks at the *same* model with verified parity, so a score delta is attributable to *process*, not capability.

2. Grader integrity is its own test. Borrowing SWE-bench's FAIL_TO_PASS / PASS_TO_PASS discipline, each task ships a hidden *gold* reference the grader must accept and a deliberately *broken* reference it must reject. The default `--self-check` mode proves all 69 graders valid (gold passes, broken fails) *before* any candidate is scored; graders test only the public contract, so a harness cannot be rewarded for matching an internal layout.

3. Spec-only isolation and contamination control. The scaffolder copies *only* each task's `TASK.md` (plus committed inputs) into an external workspace and refuses any target inside the repository, so an agent never sees graders, references, or sibling tasks. Reproducibility is pinned: Python 3.11.5, committed datasets, fixed seeds, `PYTHONHASHSEED=0`. Contamination is resisted with domain-specific entry-point names, twisted classics, and a dated `TASK.md` per task for temporal filtering.

4. A failure-frontier task design. The suite is 69 tasks across ten *weighted* categories (easy$=1$ up to competitive$=8$), so a harness cannot inflate its score by acing only the easy ones. Crucially, MiHaCoBench *discovered* that well-specified tasks under-discriminate: a careful frontier model reads the spec and one-shots them, and the arms tie. The suite therefore targets the regime where single shots fail—hard complexity gates that physically time out wrong-complexity solutions, multi-file fault localization, statistical surface-form twists where the memorized answer is wrong, and long state cascades. This finding is itself the contribution: *separating harnesses requires failure-frontier tasks, not merely well-specified ones.*

What we observe. Exploratory single-run (n=1) comparisons on a 79-task superset bear this out: the arms tie on the large majority of tasks ($72/79$), and the few that discriminate—e.g. a boundary-ambiguity pagination task that splits the no-harness baseline from the harnessed arms—are exactly the failure-frontier items the design targets. We report these as directional, not statistically significant; their value is in showing that the *instrument* localizes where harnesses diverge—precisely what a development test bench must do.

6. Conclusion

Much of the value left on the table by AI coding assistants is recoverable with *process*, not a bigger model. HarnessFlow turns a one-shot tool into a disciplined collaborator—classify, analyze in parallel, self-challenge, validate, remember—with three modes that price thoroughness explicitly (FAST matching GENERAL at half the cost). MiHaCoBench makes the claim testable by scoring the pipeline, not the model, on the failure frontier where pipelines diverge. Build the harness; measure the harness.

References

Austin, J., Odena, A., Nye, M., Bosma, M., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Jain, N., Han, K., Gu, A., Li, W.-D., et al. LiveCodeBench: Holistic and contamination-free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.